

Graphical Abstract

Using Variational Inference to Improve the Efficiency of MCMC Algorithms

Pingping Yin, Xiyun Jiao

Highlights

Using Variational Inference to Improve the Efficiency of MCMC Algorithms

Pingping Yin, Xiyun Jiao

- Research highlight 1
- Research highlight 2

Using Variational Inference to Improve the Efficiency of MCMC Algorithms

Pingping Yin, Xiyun Jiao¹

Southern University of Science and Technology, No. 1088, Xueyuan Avenue, Nanshan District, Shenzhen, 518055, Guangdong, China

Abstract

Bayesian statistics makes inference based on Bayes' theorem, but the posterior distribution of unknown parameters is typically analytically intractable. To estimate the posterior, two widely used numerical approximation methods are Markov Chain Monte Carlo (MCMC) and variational inference (VI). MCMC methods produce asymptotically exact samples but are computationally intensive, while VI methods are faster and more scalable but may lack accuracy. This paper proposes combining MCMC and VI to construct algorithms that leverage the strengths of both. The first proposed algorithm uses Gaussian variational inference (GVI) with various covariance structures to derive a linear transformation matrix for Hamiltonian Monte Carlo (HMC). This method improves the efficiency of HMC, particularly in high-dimensional and complex target distributions. The second algorithm combines a VI-based generative model, the variational auto-encoder (VAE), with the Metropolis-Hastings (MH) sampler. The resulting VAE-MH sampler is efficient and effectively traverses the parameter space, outperforming standard MCMC methods in identifying all modes of multi-modal distributions. A code implementation can be found at VAE-MH-Sampler.

Keywords:

Bayesian inference Efficiency Gaussian Variational Inference Markov Chain Monte Carlo Variational Auto-Encoder

¹Corresponding author: jiaoxy@sustech.edu.cn

1. Introduction

Bayesian statistics make inference based on the posterior distribution of the unknown parameters. Suppose $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_n)$ is the data and $\boldsymbol{\theta} \in \Theta \subset \mathbb{R}^d$ is the vector of unknown parameters. We specify the likelihood function to be $p(\mathbf{x}|\boldsymbol{\theta})$ and the prior distribution for $\boldsymbol{\theta}$ to be $p(\boldsymbol{\theta})$. Then Bayes's theorem gives the posterior distribution of $\boldsymbol{\theta}$ as

$$p(\boldsymbol{\theta}|\mathbf{x}) = \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{p(\mathbf{x})} = \frac{p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta})}{\int p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta}) d\boldsymbol{\theta}},$$

where $p(\mathbf{x}) \equiv \int p(\boldsymbol{\theta})p(\mathbf{x}|\boldsymbol{\theta}) d\boldsymbol{\theta}$ is called the *marginal likelihood* (or *evidence* sometimes). Since $p(\mathbf{x})$ is often analytically intractable in practice, the posterior distribution $p(\boldsymbol{\theta}|\mathbf{x})$ is generally not in closed form. Currently there are two widely used approaches to approximate the posterior distribution, which are Markov Chain Monte Carlo (MCMC) and variational inference (VI) respectively, each having its own strengths and weaknesses.

MCMC is a class of sampling-based methods, and it does the approximation by constructing a Markov chain $\{\boldsymbol{\theta}^{(t)}, t = 1, 2, \dots\}$ for which the stationary distribution is the target posterior $p(\boldsymbol{\theta}|\mathbf{x})$. The Metropolis-Hastings (MH) algorithm (Metropolis et al., 1953; Hastings, 1970) is one of the most commonly applied MCMC methods. MH produces samples from the target distribution by generating candidates from a proposal distribution first and accepting them with a probability. One of the most widely used MH proposals is the random walk (RW), that is, a Gaussian or uniform distribution centered around the current state $\boldsymbol{\theta}^{(t)}$. The MH algorithm with the RW proposal is easy to implement. However, due to its limitation of generating local and completely random moves, RW is not efficient for sampling high-dimensional or special-shape targets, such as multi-modal distributions. Furthermore, largely differing variances and high correlations among the parameters may exacerbate the situation.

Many new proposals have been developed to alleviate the problems related to RW. Among them, Metropolis Adjusted Langevin algorithm (MALA) and Hamiltonian Monte Carlo (HMC) are most popular in practice. Both algorithms use the gradient information of the target distribution in their proposals to generate candidates closer to high-density regions and thus avoid random sampling. MALA derives its proposal distribution from a discretization of the Langevin diffusion whose stationary distribution is the target, while HMC generates new samples via a discretized Hamiltonian dynamics,

of which the ‘position’ variable is the parameters of interest $\boldsymbol{\theta}$ and the potential energy is the negative logarithm of the target density. MALA and HMC are much more robust to dimensionality compared with RW. Asymptotically, as $n \rightarrow \infty$, the efficiencies of MALA and HMC decay by the dimension of $\boldsymbol{\theta}$ in the order of $d^{-1/3}$ and $d^{-1/4}$ respectively (Roberts and Rosenthal, 1998; Beskos et al., 2013), while that of RW scales by d^{-1} (Gelman et al., 1997).

To address the heterogeneity of scales and high correlations between variables, all of RW, MALA and HMC algorithms may employ a matrix to perform the linear transformation on the parameters. There have not been any formal principles to define such a matrix and one common choice is the Cholesky decomposition of a covariance matrix estimated from the burn-in iterations of $\boldsymbol{\theta}$. Haario et al. (1999, 2001) introduced two versions of adaptive RW Metropolis algorithms, both updating the covariance matrix of the Gaussian proposal constantly to adapt to the target distribution. While the adaptive proposal (AP) algorithm of Haario et al. (1999) updates the covariance matrix using a fixed number of previous iterations, the adaptive Metropolis (AM) algorithm of Haario et al. (2001) using all previous iterations. The theoretical arguments in Haario et al. (2001) guarantee the good statistical properties of AM and the numerical results there show its more robust performance in practice compared to the standard RW and AP. However, for high-dimensional targets, the adaption of AM becomes slower and its sensitivity to the bad choice of initial covariance increases.

Neither the standard MALA nor HMC algorithm is effective in sampling from multimodal distributions with isolated modes (see e.g., Samsonov et al. 2022 & Lan et al. 2014 for examples), mostly due to the local nature of the way they explore the parameter space. One classical strategy to deal with multimodal distributions is tempering, with well-known related MCMC algorithms to be simulated tempering (Marinari and Parisi, 1992) and parallel tempering (Geyer, 1991), both of which employ a series of ‘tempered’ target distributions, and uses the samples from ‘hot’ distributions to help the jump between modes for the original target. Park (2021) tried to incorporate tempering transitions into HMC to address sampling from high-dimensional multimodal distributions. However, designing the temperature scheme is not a trivial, even a restrictive task, for the practitioners of any of the tempering-based algorithms.

Another line of efforts to address multimodal targets is to combine local proposals with global ones. Well-designed global proposals can jump between modes efficiently while local proposals generally do better in exploring the

region around one single mode. Tak et al. (2018) introduced a new proposal consisting of both a forced downhill and a forced uphill moves, which make the local modes repelling and attracting respectively. The numerical experiments in Tak et al. (2018) show that the downhill transitions are easy to be accepted while making successful uphill moves is challenging, especially for high-dimensional problems. Independent samplers are the most common choices for global proposal, but they are hard to design. focused on developing primarily focus on designing smart global proposals. include designing special proposal distributions tailored to facilitate movement between modes. Moreover, generative models are designed to learn the unknown data distribution using the data from that distribution and generate new data that closely resemble the real data. Gabri   et al. (2021) build on works which assists a MCMC algorithm - MALA with a generative model - normalizing flow to accelerate mixing. Hunt-Smith et al. (2024) introduced an adapted Metropolis-Hastings algorithm which combines local proposals with global proposals constructed by the generative model - diffusion models.

VI is an optimization-based approach to approximating distributions (Blei et al., 2017). VI searches for the optimal distribution that minimizes the distance (typically measured by the Kullback–Leibler (KL) divergence) to the target posterior from a set of tractable distributions. Compared to MCMC, VI is much less expensive in computation and much more scalable with dimension (Robert and Casella, 2004). However, VI can introduce significant bias if all the distributions in the candidate family are far away from the target distribution (Zhang and Matsen IV, 2019). To address this limitation, normalizing flows have been introduced to expand the family of tractable distributions (Rezende and Mohamed, 2015a). By composing simple invertible transformations, normalizing flows construct flexible distributions that can approximate complex posteriors with high accuracy.

Given their complementary strengths and weaknesses, combining MCMC and VI can potentially yield more efficient algorithms. However, previous attempts to use VI approximations as proposal distributions for MH samplers have faced scalability and mixing efficiency issues.

In this paper, we propose two novel approaches that effectively combine VI and MCMC.

1. **Linear Transformation in Hamiltonian Monte Carlo algorithm via Gaussian Variational Inference:** Gaussian variational inference (GVI) with various covariance structures is used to derive a precondi-

tioning matrix for HMC, specifically, we consider GVI with Cholesky decomposed covariance (Tran et al., 2021), GVI with factor decomposed covariance (Ong et al., 2018) and GVI with sparse precision matrices (Tan and Nott, 2018). This approach addresses the challenge of computing linear transformation matrices for high-dimensional models (Girolami and Calderhead, 2011). Tests show that HMC with GVI-derived linear transformation matrices outperforms standard MCMC algorithms, especially as the dimension and complexity of the target distribution increase.

2. **VAE-MH Sampler:** A VAE-type generative model (Kingma and Welling, 2014) is used to construct a global proposal distribution for an MH sampler (Hastings, 1970). The implementation includes standard VAEs, flow-augmented VAEs, component-conditioned GMM-VAEs, and Student- t decoder variants for heavy-tailed targets. This “VAE-MH” sampler is easy to implement and more efficient in exploring the parameter space than state-of-the-art MCMC algorithms. Tests on multi-dimensional mixture Gaussian and mixture t distributions with distant modes demonstrate that VAE-MH can successfully identify all modes of a multi-modal distribution, whereas MH often gets trapped in local modes.

The paper is organized as follows. Section 2 provides an overview of MCMC and VI algorithms. Section 3 introduces two novel approaches that effectively combine VI and MCMC: (1) linear transformation in HMC via GVI, and (2) a generative-proposal MH sampler that uses VAE-type models to address multi-modal sampling. Section 4 presents numerical studies evaluating both methods across various challenging distributions. The paper concludes with a summary and discussion in Section 5.

2. Preliminaries

2.1. Hamiltonian Monte Carlo algorithm

HMC improves upon RW by augmenting the target with auxiliary momentum variables $\boldsymbol{\psi}$ and using discretized Hamiltonian dynamics to generate distant proposals (Neal, 2011). Given a positive definite mass matrix $\mathbf{M}$, the Hamiltonian is $H(\boldsymbol{\theta}, \boldsymbol{\psi}) = U(\boldsymbol{\theta}) + K(\boldsymbol{\psi})$, where $U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta})$, $K(\boldsymbol{\psi}) = \frac{1}{2}\boldsymbol{\psi}^\top \mathbf{M}^{-1}\boldsymbol{\psi}$. The corresponding joint density is proportional to $\exp\{-H(\boldsymbol{\theta}, \boldsymbol{\psi})\} = p(\boldsymbol{\theta}) \exp\{-K(\boldsymbol{\psi})\}$, so that the marginal density of $\boldsymbol{\theta}$ is the desired target.

The Hamiltonian dynamics are governed by $\frac{d\boldsymbol{\theta}}{dt} = \mathbf{M}^{-1}\boldsymbol{\psi}$, $\frac{d\boldsymbol{\psi}}{dt} = \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta})$. Since these differential equations are rarely solvable in closed form, HMC uses a reversible and volume-preserving leapfrog integrator. One leapfrog step with step size ϵ is

$$\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}), \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \mathbf{M}^{-1} \boldsymbol{\psi}, \quad \boldsymbol{\psi} \leftarrow \boldsymbol{\psi} + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}).$$

After L leapfrog steps, the proposal $(\boldsymbol{\theta}^*, \boldsymbol{\psi}^*)$ is accepted with probability

$$\alpha_{\text{HMC}} = \min \{1, \exp [-H(\boldsymbol{\theta}^*, \boldsymbol{\psi}^*) + H(\boldsymbol{\theta}, \boldsymbol{\psi})]\}.$$

The exact Hamiltonian flow preserves energy and volume, while the leapfrog integrator preserves reversibility and volume but introduces a small discretization error. The final MH correction removes this numerical bias. Compared with RMH, HMC uses gradient information to propose larger moves while maintaining reasonable acceptance probabilities. Its efficiency, however, depends on the step size ϵ , the number of leapfrog steps L , and especially the mass matrix $\mathbf{M}$. Poor choices of $\mathbf{M}$ can lead to slow exploration or require very small step sizes for stable integration. This motivates the covariance-based preconditioning strategy developed in Section 3.1.

2.2. Gaussian variational inference

GVI seeks the closest distribution to the target posterior within a Gaussian family, i.e. the variational distribution $q_{\boldsymbol{\lambda}}(\boldsymbol{\theta})$ is a Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$. Here we explore several variants of GVI and introduce the strength and applications of each.

GVI with Cholesky decomposition (CGVI) uses the Cholesky decomposition for the covariance matrix $\boldsymbol{\Sigma}$ of the variational distribution $q_{\boldsymbol{\lambda}}(\boldsymbol{\theta})$ (Tran et al., 2021). Specifically, $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is a lower triangular matrix. By utilizing the reparameterization trick (Kingma and Welling, 2014), it reduce the variance of the gradient estimators. For high-dimensional variational distributions, the computational complexity of CGVI can be prohibitive. An alternative is GVI with factor decomposed covariance (FGVI) (Ong et al., 2018), which uses the factor decomposition $\boldsymbol{\Sigma} = \mathbf{B}\mathbf{B}^\top + \mathbf{C}^2$. Here, $\mathbf{B}$ is a factor loading matrix and $\mathbf{C}$ is a diagonal matrix. This reduces the number of variational parameters significantly. GVI with sparse precision matrices, proposed by Tan and Nott (2018), assumes a multivariate Gaussian distribution with a sparse precision matrix $\boldsymbol{\Omega} = \mathbf{L}\mathbf{L}^\top$. Imposing sparsity on $\mathbf{L}$ achieves conditional independence in $\boldsymbol{\Omega}$. The full GVI algorithms are detailed in papers (Tran et al., 2021; Ong et al., 2018; Tan and Nott, 2018).

2.3. Variational Auto-Encoders and Multi-modal Extensions

VAEs provide a flexible class of latent-variable generative models for approximating complex data distributions. In the standard formulation, the marginal model is written as $p_\theta(x) = \int p_\theta(x | z)p(z) dz$, where $p(z)$ is usually chosen as a standard Gaussian prior and $p_\theta(x | z)$ is a decoder distribution parameterized by a neural network. Since the posterior $p_\theta(z | x)$ is generally intractable, VAEs introduce an amortized variational approximation $q_\phi(z | x)$ and maximize the evidence lower bound $\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL} \{q_\phi(z | x) \| p(z)\}$. The reparameterization trick provides a differentiable stochastic estimator of this objective, allowing the encoder and decoder parameters to be optimized jointly (Kingma and Welling, 2014).

The basic VAE is computationally attractive, but its common use of a unimodal Gaussian prior and a factorized Gaussian variational posterior can be restrictive for distributions with separated modes. One line of work addresses this limitation by increasing the flexibility of the variational posterior. Normalizing flows transform a simple base density through a sequence of invertible maps,

$$z_K = f_K \circ \dots \circ f_1(z_0), \quad \log q_K(z_K) = \log q_0(z_0) - \sum_{k=1}^K \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|. \quad (1)$$

This construction gives a richer posterior family while keeping density evaluation tractable through the change-of-variables formula (Rezende and Mohamed, 2015b). However, improving the posterior alone does not necessarily solve the generative mismatch caused by a simple prior. A complementary direction is to introduce mixture structure into the latent generative model. Conditional-prior VAEs, for example, use a discrete latent variable $c \in \{1, \dots, K\}$ together with a continuous latent variable z , yielding $p_\theta(x) = \sum_{c=1}^K \int p_\theta(x | z, c)p_\phi(z | c)p(c) dz$. The corresponding inference model is often factorized as $q_\phi(z, c | x) = q_\phi(z | x, c)q_\phi(c | x)$, and the KL term decomposes into a categorical regularization term and a component-wise continuous latent regularization term. (Lavda et al., 2020). Related mixture-based extensions include deep latent Gaussian mixture models, where mixture weights are treated as latent variables and handled through stick-breaking approximations (Nalisnick et al., 2016); mixtures of VAEs, where different VAE components are trained jointly with regularization to discourage redundant components (Ye and Bors, 2020); and hierarchical compositional mixtures,

which combine VAE leaves with sum-product network structures to decompose high-dimensional density estimation into smaller local problems (Tan and Peharz, 2019). In Section 3.2.2, we use these ideas to construct VAE-based proposal families for the proposed Metropolis–Hastings sampler.

3. Methodology

3.1. Linear Transformation in HMC via GVI

HMC is invariant under nonsingular linear transformations. This property provides useful guidance for choosing the momentum distribution when the target distribution has strong scale differences or correlations. Let $p(\boldsymbol{\theta})$ denote the unnormalized target density on $\mathbb{R}^d$, and let $\ell(\boldsymbol{\theta}) = \log p(\boldsymbol{\theta})$. Suppose that the target is approximately Gaussian with covariance matrix $\boldsymbol{\Sigma}$. If $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$, then the whitening transformation $\boldsymbol{\eta} = \mathbf{L}^{-1}(\boldsymbol{\theta} - \boldsymbol{\mu})$ maps the approximately correlated distribution in the original $\boldsymbol{\theta}$ -space to an approximately isotropic distribution in the transformed $\boldsymbol{\eta}$ -space.

Running isotropic HMC in the whitened coordinates is equivalent to running HMC in the original coordinates with a non-isotropic momentum distribution. To see this, write $\boldsymbol{\theta} = \boldsymbol{\mu} + \mathbf{L}\boldsymbol{\eta}$. The corresponding canonical transformation of the momentum variables is $\boldsymbol{\psi} = \mathbf{L}^{-\top}\boldsymbol{\rho}$, where $\boldsymbol{\rho}$ is the momentum variable in the whitened coordinate system. If $\boldsymbol{\rho} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then $\boldsymbol{\psi} \sim \mathcal{N}(\mathbf{0}, \mathbf{L}^{-\top}\mathbf{L}^{-1}) = \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}^{-1})$. The kinetic energy in the original coordinates is therefore $K(\boldsymbol{\psi}) = \frac{1}{2}\boldsymbol{\psi}^\top \boldsymbol{\Sigma} \boldsymbol{\psi}$. Equivalently, the Hamiltonian can be written as $H(\boldsymbol{\theta}, \boldsymbol{\psi}) = -\ell(\boldsymbol{\theta}) + \frac{1}{2}\boldsymbol{\psi}^\top \boldsymbol{\Sigma} \boldsymbol{\psi}$. The corresponding Hamiltonian equations are $\frac{d\boldsymbol{\theta}}{dt} = \nabla_{\boldsymbol{\psi}} K(\boldsymbol{\psi}) = \boldsymbol{\Sigma} \boldsymbol{\psi}$, $\frac{d\boldsymbol{\psi}}{dt} = \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta})$. Thus, when the momentum covariance is chosen as $\boldsymbol{\Sigma}^{-1}$, the leapfrog position update in the original parameterization is $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \epsilon \boldsymbol{\Sigma} \boldsymbol{\psi}$.

In practice, the true target covariance is unknown. We therefore use GVI to obtain a Gaussian approximation $q_{\text{GVI}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}_{\text{GVI}}, \boldsymbol{\Sigma}_{\text{GVI}})$ to the target distribution. The GVI mean is used as the initial value of the Markov chain, and the GVI covariance is used as a fixed estimate of the position covariance, $\mathbf{S} = \boldsymbol{\Sigma}_{\text{GVI}}$. We then sample the momentum from $\boldsymbol{\psi} \sim \mathcal{N}(\mathbf{0}, \mathbf{S}^{-1})$ and use the kinetic energy $K(\boldsymbol{\psi}) = \frac{1}{2}\boldsymbol{\psi}^\top \mathbf{S} \boldsymbol{\psi}$. For any fixed positive definite matrix $\mathbf{S}$, the Metropolis–Hastings correction preserves the target distribution. The accuracy of $\mathbf{S}$ affects the efficiency of the sampler, but not the validity of the resulting Markov chain.

The resulting GVI-preconditioned HMC algorithm is given in Algorithm 1.

For comparison, we also consider a burn-in-based covariance estimate. In that version, a preliminary HMC run is first performed with a simple choice such as $\mathbf{S} = \mathbf{I}_d$. The empirical covariance matrix of the preliminary samples is then used as $\mathbf{S}_{\text{burn}} = \widehat{\text{Cov}}(\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(B_0)})$. The main HMC run is subsequently performed using $\mathbf{S} = \mathbf{S}_{\text{burn}}$ in Algorithm 1. In both the GVI-based and burn-in-based versions, the matrix $\mathbf{S}$ is fixed during the main sampling stage. The step size ϵ and the number of leapfrog steps L are selected by short preliminary runs, with the aim of obtaining stable acceptance probabilities and reducing autocorrelation in the resulting Markov chain.

3.2. Generative-Proposal Metropolis-Hastings for Multi-modal Targets

Sampling from a multi-modal target density $p(\boldsymbol{\theta})$ is difficult for three related reasons. First, the sampler must identify high-probability regions where the modes are located. Second, it must move between these regions by crossing low-probability barriers. Third, once a mode has been reached, it must sample efficiently within that local region while accounting for differences in scale, shape, and local geometry. We address these three tasks using a generative-proposal MH sampler. The resulting method combines a mode-finding stage, a VAE-based global proposal, a kernel-density MH correction, and a local random-walk kernel.

3.2.1. Identifying high-probability regions

Before training a generative proposal, we need an initial set of samples that covers the important regions of the target. The classic Crowding-based Differential Evolution (CrowdingDE) algorithm (Thomsen, 2004)—a simple yet efficient population-based approach from the evolutionary computation community—is employed to identify the peak locations of the multi-modal target distributions by maximizing its probability density function. This approach offers a distinct advantage over classical optimization techniques, which often require multiple restarts without guaranteeing diverse solutions. For further details, please refer to Storn and Price (1997) and the recent survey Chauhan et al. (2025).

The complete pseudo-code is provided in Algorithm 2. Parameter settings follow the original reference: population size $N = 100$, mutation factor $F = 0.5$, crossover rate $CR = 0.9$, and the maximum number of generations is set to $G_{\text{max}} = 10000$. The search space dimensionality is defined by the dimension of targets. All other parameters remain consistent with the original implementation.

Let $\hat{\mathbf{m}}_1, \dots, \hat{\mathbf{m}}_J$ denote the estimated mode locations. After obtaining the final CrowdingDE population, we cluster the individuals by Euclidean distance and select, from each cluster, the individual with the largest objective value as the representative mode. We then generate an initial training set $\mathcal{D}_0 = \bigcup_{j=1}^J \{\boldsymbol{\theta}_{j\ell} : \boldsymbol{\theta}_{j\ell} \sim \mathcal{N}(\hat{\mathbf{m}}_j, s_0^2 \mathbf{I}), \ell = 1, \dots, n_0\}$, where s_0 is a local scale parameter. This initialization gives the generative model information about all relevant high-probability regions before the MCMC chain begins.

3.2.2. Learning a generative proposal family

At adaptation round r , the available samples in the parameter space are collected into a training set $\mathcal{D}_r = \{\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_{n_r}\}$. In this step, the parameter vector $\boldsymbol{\theta}$ is treated as the observation variable in a VAE-type generative model. The fitted model is then used to generate global proposal candidates directly in the original parameter space.

We consider four VAE-based proposal families. The first is the standard VAE, which uses $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, $q_\lambda(\mathbf{z} \mid \boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}_\lambda(\boldsymbol{\theta}), \text{diag}\{\boldsymbol{\sigma}_\lambda^2(\boldsymbol{\theta})\})$, together with a decoder $p_\xi(\boldsymbol{\theta} \mid \mathbf{z})$. After training, proposal candidates are generated by ancestral sampling: draw $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and decode it to obtain $\boldsymbol{\theta}^*$. This serves as the baseline generative proposal. The second proposal family is a flow-augmented VAE. It replaces the diagonal Gaussian encoder with a normalizing-flow posterior of the form in (1). In our implementation, the flow transformation can be chosen from RealNVP affine coupling layers, masked autoregressive flows, planar flows, or neural spline flows. These transformations are used on the encoder side during training, thereby increasing the flexibility of the variational posterior and improving the learned decoder when the training samples exhibit nonlinear dependence. At the proposal-generation stage, candidates are still produced by sampling from the latent prior and decoding to the parameter space. The third proposal family is a component-conditioned GMM-VAE, designed for targets with separated modes. It introduces a discrete component indicator $c \in \{1, \dots, K\}$ and specifies the latent generative model as

$$c \sim \text{Categorical}(\boldsymbol{\pi}), \quad \mathbf{z} \mid c \sim \mathcal{N}(\boldsymbol{\mu}_c, \text{diag}(\boldsymbol{\sigma}_c^2)), \quad \boldsymbol{\theta} \mid \mathbf{z}, c \sim p_\xi(\boldsymbol{\theta} \mid \mathbf{z}, c). \quad (2)$$

The inference model factorizes as $q_\lambda(\mathbf{z}, c \mid \boldsymbol{\theta}) = q_\lambda(c \mid \boldsymbol{\theta})q_\lambda(\mathbf{z} \mid \boldsymbol{\theta}, c)$. The decoder is conditioned on both the continuous latent variable $\mathbf{z}$ and the component label c . This structure allows different components to specialize to different regions of the target distribution and is therefore well suited to

multimodal sampling.

Finally, for heavy-tailed targets, we replace the Gaussian decoder likelihood with a Student t decoder likelihood. Specifically, the reconstruction model is taken to be $p_{\xi}(\boldsymbol{\theta} \mid \mathbf{z}, c) = t_{\nu}(\boldsymbol{\theta} \mid \mathbf{m}_{\xi}(\mathbf{z}, c), s^2 \mathbf{I})$, with fixed degrees of freedom ν and scale s . This modification is used when the target has heavier tails than a Gaussian mixture, since it reduces the mismatch between the decoder likelihood and the empirical tail behaviour of the training samples.

Across all variants, the learned VAE-type model is used only as a mechanism for generating global candidates. The validity of the resulting sampler is retained by the subsequent Metropolis–Hastings correction, while the quality of the generative proposal affects mixing efficiency and mode exploration.

3.2.3. Moving between modes with KDE-corrected global proposals

After training a generative model G_r on $\mathcal{D}_r$, we draw a proposal pool $\mathcal{Y}_r = \{\mathbf{y}_1, \dots, \mathbf{y}_M\}$, $\mathbf{y}_m \sim G_r$. Because the marginal proposal density induced by sampling latent variables and decoding them is generally not available in closed form, we construct an explicit proposal density by applying kernel density estimation (KDE) to the generated pool:

$$q_r(\boldsymbol{\theta}) = \frac{1}{M} \sum_{m=1}^M \varphi_h(\boldsymbol{\theta} - \mathbf{y}_m), \quad (3)$$

where φ_h is a multivariate Gaussian kernel with bandwidth h . The global proposal is independent of the current state: to propose $\boldsymbol{\theta}'$, we select one generated point from $\mathcal{Y}_r$ and add Gaussian kernel noise with bandwidth h . The MH acceptance probability is then

$$\alpha_g(\boldsymbol{\theta}, \boldsymbol{\theta}') = \min \left\{ 1, \frac{\pi(\boldsymbol{\theta}') q_r(\boldsymbol{\theta})}{\pi(\boldsymbol{\theta}) q_r(\boldsymbol{\theta}')} \right\}. \quad (4)$$

This correction is important: the VAE is used to construct a useful global proposal, while the MH step corrects for the discrepancy between the learned proposal and the true target density. Conditional on a fixed proposal density q_r , the global MH kernel satisfies detailed balance with respect to π .

3.2.4. Sampling efficiently within modes

A global proposal is useful for jumping between modes, but it is not always the most efficient mechanism for local exploration. We therefore combine it with a Gaussian random-walk proposal. At iteration t , with probability ρ

we use the KDE-corrected global proposal in equation (4); with probability $1 - \rho$, we propose $\boldsymbol{\theta}' = \boldsymbol{\theta}^{(t)} + \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{rw}}^2 \mathbf{I})$, and accept it with the usual random-walk MH probability $\alpha_{\text{rw}}(\boldsymbol{\theta}, \boldsymbol{\theta}') = \min \left\{ 1, \frac{p(\boldsymbol{\theta}')}{p(\boldsymbol{\theta})} \right\}$. The local kernel improves exploration within individual modes, while the global kernel prevents the chain from being trapped in one mode. During sampling, the generative model is retrained every τ iterations using a moving window of recent chain states, so that the proposal pool can adapt as the chain explores the target. Since the proposal is updated during this adaptive stage, the resulting chain should be interpreted as an adaptive MCMC scheme. The complete procedure is summarized in Algorithm 3.

4. Numerical Studies

4.1. Linear Transformations in HMC via GVI

We compare five MCMC algorithms: RW with scale matrix estimated from burn-in samples (RMH), AM, preconditioned MALA, HMC with linear transformation matrix estimated from burn-in samples (HMC), and HMC with linear transformation matrix estimated via GVI (HMC-GVI). For each algorithm, we employ standard hyper-parameter tuning procedures to ensure fair comparison. Each algorithm runs 10^6 iterations with 10^4 burn-in iterations. Each result is an average over 3 replicated runs.

4.1.1. Posterior sampling in logistic regression models

Let $\mathbf{y} = (y_1, \dots, y_n)^\top$ be a vector of observed binary responses and $\mathbf{X}$ the corresponding design matrix with rows $\mathbf{x}_i = (1, x_{i2}, \dots, x_{id})^\top$ for $i = 1, 2, \dots, n$. All predictors are standardized to improve mixing. The generalized linear model is $(y_i | p_i) \stackrel{\text{ind}}{\sim} \text{Bernoulli}(p_i)$, $p_i = g(\boldsymbol{\beta}^\top \mathbf{x}_i)$, where $g(\cdot)$ is the inverse logit transform, i.e., $g(\eta) = e^\eta / (1 + e^\eta)$. The log-likelihood function is: $l(\boldsymbol{\beta}) = \sum_{i=1}^n \left[y_i \boldsymbol{\beta}^\top \mathbf{x}_i - \log \left(1 + e^{\boldsymbol{\beta}^\top \mathbf{x}_i} \right) \right]$. We specify a vague prior $\boldsymbol{\beta} \sim \mathcal{N}_d(\mathbf{0}, 100\mathbf{I}_d)$, yielding the posterior: $p(\boldsymbol{\beta} | \mathbf{y}) \propto l(\boldsymbol{\beta}) \times p(\boldsymbol{\beta})$. Two datasets are used: the Pima Indian dataset ($d = 8, n = 532$) and the German credit dataset ($d = 24, n = 1000$).

For these moderate-dimensional problems, we use CGVI to derive Gaussian approximations for HMC linear transformation. The CGVI running times (0.4233s and 1.6833s) are comparable to HMC burn-in times (2.8600s and 11.1700s).

Table 1: Comparison of five MCMC algorithms on Pima Indian dataset ($d = 8, n = 532$).

Method	Time(s)	E	P_{jump}	ρ_1
RMH	14.7400	0.0380	0.2392	0.9098
AM	23.4400	0.0390	0.2552	0.9234
MALA	24.2800	0.2995	0.5546	0.5029
HMC	153.9700	0.9386	0.9933	-0.0179
HMC-GVI	141.0400	1.0069	0.9931	0.0016

Table 2: Comparison of five MCMC algorithms on German credit dataset ($d = 24, n = 1000$).

Method	Time(s)	E	P_{jump}	ρ_1
RMH	24.1200	0.0122	0.2371	0.9590
AM	42.4200	0.0123	0.2245	0.9592
MALA	50.4900	0.1735	0.5738	0.6881
HMC	412.6300	0.9445	0.9877	0.0038
HMC-GVI	333.5400	1.1647	0.9859	0.0054

Tables 1 and 2 present performance metrics averaged over all parameters. Results show that GVI provides more accurate covariance estimation with less computational overhead, leading to enhanced efficiency: $E = 1.0069$ and $E = 1.1647$ for HMC-GVI versus $E = 0.9386$ and $E = 0.9445$ for standard HMC.

Figure 1 displays trace and ACF plots for β_1 in the Pima Indian dataset. MALA, HMC, and HMC-GVI demonstrate superior mixing compared to RMH and AM, with faster autocorrelation decay. HMC-GVI shows the best performance, achieving near-optimal efficiency and minimal autocorrelation.

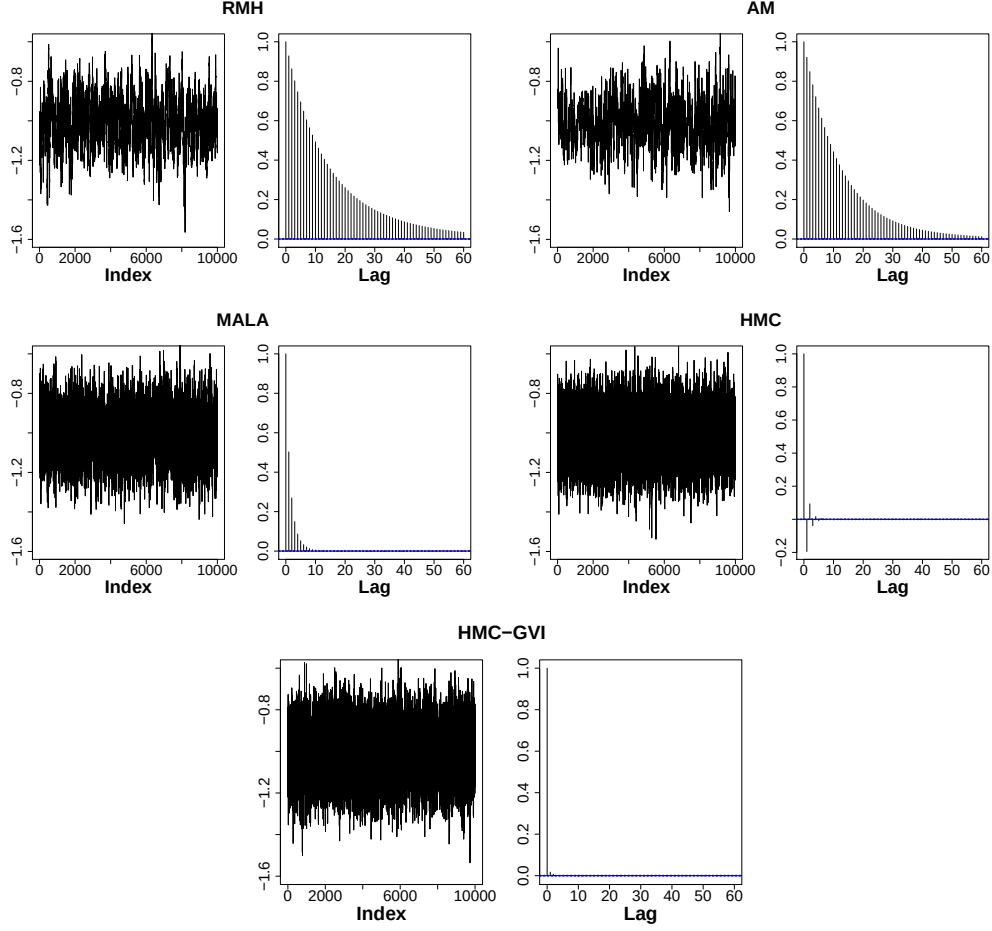


Figure 1: Trace and ACF plots of β_1 for five MCMC algorithms on Pima Indian dataset.

4.1.2. Sampling from a 100-dimensional Gaussian distribution

We next consider a 100-dimensional Gaussian target distribution,

$$p(\boldsymbol{\theta}) \propto \exp \left\{ -\frac{1}{2} \boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\theta} \right\}, \quad \boldsymbol{\theta} \in \mathbb{R}^{100},$$

where the mean vector is zero and the covariance matrix is denoted by $\boldsymbol{\Sigma}$. The diagonal entries of $\boldsymbol{\Sigma}$ are independently generated from a Gamma distribution with shape parameter 2 and scale parameter 3, while the off-diagonal entries are set to 0.8. Only positive definite realizations of $\boldsymbol{\Sigma}$ are retained. The corresponding log-density, up to an additive constant, is given by $\ell(\boldsymbol{\theta}) = -\frac{1}{2} \boldsymbol{\theta}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\theta}$, with gradient $\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) = -\boldsymbol{\Sigma}^{-1} \boldsymbol{\theta}$. This gradient is used in the

implementation of the gradient-based methods, including GVI, MALA, and HMC.

For the HMC algorithm with linear transformation via GVI, we employ the FGVI with $f = 5$ to compute the covariance matrix. The running time of FGVI is 9.6100s, and that of the burn-in for HMC is 37.6100s. The running time, efficiency (E), average acceptance rate (P_{jump}) and lag-1 autocorrelation (ρ_1) are shown for each algorithm in table 3.

Table 3: Comparison of five MCMC algorithms for sampling a 100-dimensional Gaussian distribution. E : Effective Sample Size per second (ESS/s); P_{jump} : Average jump acceptance rate; ρ_1 : Lag-1 autocorrelation of the first component.

Method	Time (s)	E (ESS/s)	P_{jump}	ρ_1
AM	497.08	0.00	0.37	0.99
MALA	309.68	0.07	0.57	0.80
HMC	3442.68	0.88	0.98	0.01
HMC-GVI	3240.99	1.01	0.97	-0.01

The results in table 3 show that for high-dimensional target distributions, the running time of FGVI is significantly shorter than that of the burn-in for HMC. However, FGVI provides a more accurate estimate of the covariance matrix than the burn-in samples of MCMC, and this improved accuracy leads to an MCMC algorithm with higher efficiency, as demonstrated by that fact that $E = 1.0050$ for the HMC with linear transformation via FGVI while $E = 0.8825$ for the HMC with transformation matrix estimated by the burn-in samples.

4.1.3. Posterior sampling in a 509-dimensional GLMM

We finally consider posterior sampling for a high-dimensional generalized linear mixed model (GLMM). Let y_{ij} denote the binary response for subject $i = 1, \dots, n$ at occasion $j = 1, \dots, n_i$. Conditional on the subject-specific random effect b_i , we assume $y_{ij} \mid b_i, \boldsymbol{\beta} \sim \text{Bernoulli}(\mu_{ij})$, $\text{logit}(\mu_{ij}) = X_{ij}^\top \boldsymbol{\beta} + Z_{ij}^\top b_i$, where $\boldsymbol{\beta}$ denotes the fixed-effect coefficients and b_i denotes the random effect for subject i . The random effects are assumed to satisfy $b_i \mid \boldsymbol{\xi} \sim N(\mathbf{0}, \mathbf{G})$, $\mathbf{G} = \mathbf{L}\mathbf{L}^\top$, where $\mathbf{L}$ is the Cholesky factor of $\mathbf{G}$. To ensure an unconstrained parametrization, we work with the transformed Cholesky factor $\mathbf{L}^*$, defined by $L_{kk}^* = \log(L_{kk})$, $L_{kl}^* = L_{kl}$ ($k \neq l$), and set $\boldsymbol{\xi} = \text{vech}(\mathbf{L}^*)$. The priors are specified as $\boldsymbol{\beta} \sim N(\mathbf{0}, \sigma_\beta^2 \mathbf{I})$, $\boldsymbol{\xi} \sim N(\mathbf{0}, \sigma_\xi^2 \mathbf{I})$.

Let $\boldsymbol{\theta} = (\boldsymbol{\beta}^\top, b_1^\top, \dots, b_n^\top, \boldsymbol{\xi}^\top)^\top$ denote all unknown parameters. Up to a normalizing constant, the posterior density is

$$\pi(\boldsymbol{\theta} \mid \mathbf{y}) \propto p(\boldsymbol{\beta})p(\boldsymbol{\xi}) \prod_{i=1}^n \left\{ p(b_i \mid \boldsymbol{\xi}) \prod_{j=1}^{n_i} p(y_{ij} \mid \boldsymbol{\beta}, b_i) \right\}.$$

Equivalently, the log-posterior density, up to an additive constant, is

$$\begin{aligned} \log \pi(\boldsymbol{\theta} \mid \mathbf{y}) = & \sum_{i=1}^n \sum_{j=1}^{n_i} [y_{ij}\eta_{ij} - h_1(\eta_{ij})] - n \log |\mathbf{L}| - \frac{1}{2} \sum_{i=1}^n b_i^\top \mathbf{L}^{-\top} \mathbf{L}^{-1} b_i \\ & - \frac{1}{2\sigma_\beta^2} \boldsymbol{\beta}^\top \boldsymbol{\beta} - \frac{1}{2\sigma_\xi^2} \boldsymbol{\xi}^\top \boldsymbol{\xi}, \end{aligned} \quad (5)$$

where $\eta_{ij} = X_{ij}^\top \boldsymbol{\beta} + Z_{ij}^\top b_i$, $h_1(x) = \log\{1 + \exp(x)\}$.

The gradient of the log-posterior is required for the implementation of gradient-based samplers. For $i = 1, \dots, n$,

$$\nabla_{b_i} \log \pi(\boldsymbol{\theta} \mid \mathbf{y}) = \sum_{j=1}^{n_i} \{y_{ij} - h'_1(\eta_{ij})\} Z_{ij} - \mathbf{L}^{-\top} \mathbf{L}^{-1} b_i,$$

and

$$\nabla_{\boldsymbol{\beta}} \log \pi(\boldsymbol{\theta} \mid \mathbf{y}) = \sum_{i=1}^n \sum_{j=1}^{n_i} \{y_{ij} - h'_1(\eta_{ij})\} X_{ij} - \frac{1}{\sigma_\beta^2} \boldsymbol{\beta}.$$

The gradient with respect to $\boldsymbol{\xi}$ is

$$\nabla_{\boldsymbol{\xi}} \log \pi(\boldsymbol{\theta} \mid \mathbf{y}) = -n \mathbf{1}_{\text{diag}(\mathbf{L})} + \mathbf{1}_{\boldsymbol{\xi}} \odot \text{vech}(\mathbf{A}) - \frac{1}{\sigma_\xi^2} \boldsymbol{\xi},$$

where $\mathbf{A} = \sum_{i=1}^n \mathbf{L}^{-\top} \mathbf{L}^{-1} b_i b_i^\top \mathbf{L}^{-\top}$. Here, the k th element of $\mathbf{1}_{\boldsymbol{\xi}}$ is $\exp(\xi_k)$ if ξ_k corresponds to a diagonal element of $\mathbf{L}$, and is one otherwise. Similarly, the k th element of $\mathbf{1}_{\text{diag}(\mathbf{L})}$ is one if ξ_k corresponds to a diagonal element of $\mathbf{L}$, and is zero otherwise.

We use the polypharmacy data set from the R package `aplore3`. Following Tan and Nott (2018), we fit the logistic random-intercept model

$$\begin{aligned} \text{logit}(\mu_{ij}) = & \beta_0 + \beta_1 \text{Gender}_i + \beta_2 \text{Race}_i + \beta_3 \text{Age}_{ij} + \beta_4 \text{MHV1}_{ij} \\ & + \beta_5 \text{MHV2}_{ij} + \beta_6 \text{MHV3}_{ij} + \beta_7 \text{INPTMHV}_{ij} + b_i, \end{aligned}$$

where $i = 1, \dots, 500$ and $j = 1, \dots, 7$. The response indicates whether the subject used drugs from three or more distinct groups. The covariates are gender, race, age, indicators for outpatient mental-health visits MHV1, MHV2, and MHV3, and an indicator for inpatient mental-health visits INPTMHV. Specifically, Gender is coded as one for male and zero for female, Race is coded as zero for white and one for other races, MHV1 = 1 indicates one to five outpatient mental-health visits, MHV2 = 1 indicates six to fourteen visits, MHV3 = 1 indicates fifteen or more visits, and INPTMHV = 1 indicates at least one inpatient mental-health visit.

In this application, there are eight fixed-effect coefficients and 500 subject-specific random intercepts. Since the random-effect covariance reduces to a scalar variance component, ξ is one-dimensional. Thus, the full parameter vector has dimension $8 + 500 + 1 = 509$. We assign the priors $\beta \sim N(\mathbf{0}, 100\mathbf{I}_8)$, $\xi \sim N(0, 100)$, $b_i \mid \xi \sim N(0, \exp(2\xi))$. The posterior density is then given by (5).

To sample from the posterior distribution of the GLMM for the polypharmacy dataset, we compare the performance of four algorithms: preconditioned MALA with the preconditioning matrix estimated from preliminary MALA samples, preconditioned MALA with the matrix estimated via Gaussian variational inference (GVI), HMC with a linear transformation matrix estimated from preliminary HMC samples, and HMC with the matrix estimated via GVI. We exclude the random-walk MH and AM algorithms due to their low sampling efficiency. For each algorithm, we tune the hyperparameters following the guidelines in the MCMC literature to achieve optimal sampling efficiency, and run a Markov chain of 10^6 iterations with a burn-in period of 10^4 .

Due to the complex nature of the target distribution in this experiment, we use the GVI with sparse precision matrix proposed by Tan et al. (2018) Tan and Nott (2018) to estimate the covariance matrix which is used in both the MALA and HMC algorithms. For the polypharmacy dataset, the running time of GVI with sparse precision matrix is approximately 36.5733s. Since the model here is more complicated than the models before, for estimating the covariance matrix with preliminary samples of the MCMC algorithms, we run the MALA for 85000 iterations, taking around 37.29s, comparable to the time spent by GVI, and we run HMC for 10^5 iterations, taking around 497.80s.

The running time, efficiency (E), average acceptance rate (P_{jump}) and lag-1 autocorrelation (ρ_1) are shown for each algorithm in table 4. Each

result is an average over 3 replicated runs.

Table 4: Comparison of four MCMC algorithms on the polypharmacy dataset under the 509-dimensional GLMM model.

Method	Time(s)	E	P_{jump}	ρ_1
MALA	425.9000	0.0008	0.5893	0.9944
MALA-GVI	272.9900	0.0296	0.5742	0.9352
HMC	2782.6600	0.7125	0.9292	0.0041
HMC-GVI	2708.3900	0.8414	0.9333	-0.0083

From the results we can see that for sampling from high-dimensional and complex target distributions, deriving the transformation matrix in the MALA and HMC algorithms using GVI is beneficial. First, the running time of GVI is significantly shorter than that of the preliminary runs of MCMC. Furthermore, GVI yields a more accurate estimate of the covariance matrix and this improvement in accuracy leads to the increase in efficiency of the corresponding MCMC algorithms. Specifically, the efficiencies are 0.0296 and 0.8414 for the MALA and HMC respectively with transformation matrix estimated with GVI, while 0.0008 and 0.7125 respectively for the algorithms with transformation matrix estimated using the preliminary MCMC samples.

4.2. Efficient Bayesian Sampling Using the VAE-MH Algorithm

We evaluate the sampling efficiency of the proposed VAE-MH algorithm on multimodal target distributions with different dimensions, tail behaviours, and numbers of modes. We compare VAE-MH with RMH and Diffusion-MH. Each sampler is run for 100,000 iterations, with the first 10,000 iterations discarded as burn-in. Visual diagnostics are provided through corner plots for higher-dimensional examples and trajectory or sample plots for two-dimensional examples. Numerical performance is summarized by running time (Time), effective sample size per second (ESS/s), sampling efficiency (Efficiency), mode coverage (Mode cov.), mode-frequency total variation distance (Mode TV), and empirical mode-transition rate (Mode trans.). All the reported values are averages over five independent runs.

4.2.1. Sampling mixture of Gaussian and mixture of Student t

We first evaluate the proposed VAE-MH algorithm on two low-dimensional multimodal targets, following the experimental setting of Habib and Bar-

ber (2019). Specifically, let $\boldsymbol{\mu}_1 = (10, 0)^\top$, $\boldsymbol{\mu}_2 = (-10, 0)^\top$, $\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \mathbf{I}_2$, so that the Euclidean distance between the two component means is 20. The two-component Gaussian mixture target is defined as $p_G(\mathbf{x}) = \omega \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + (1 - \omega) \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$, with $\omega = 1/2$.

We also consider a heavier-tailed analogue based on Student t components. Let $\nu = 5$ denote the degrees of freedom. The Student t mixture target is $p_T(\mathbf{x}) = \omega t_\nu(\mathbf{x} | \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + (1 - \omega) t_\nu(\mathbf{x} | \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$ with $\omega = 1/2$.

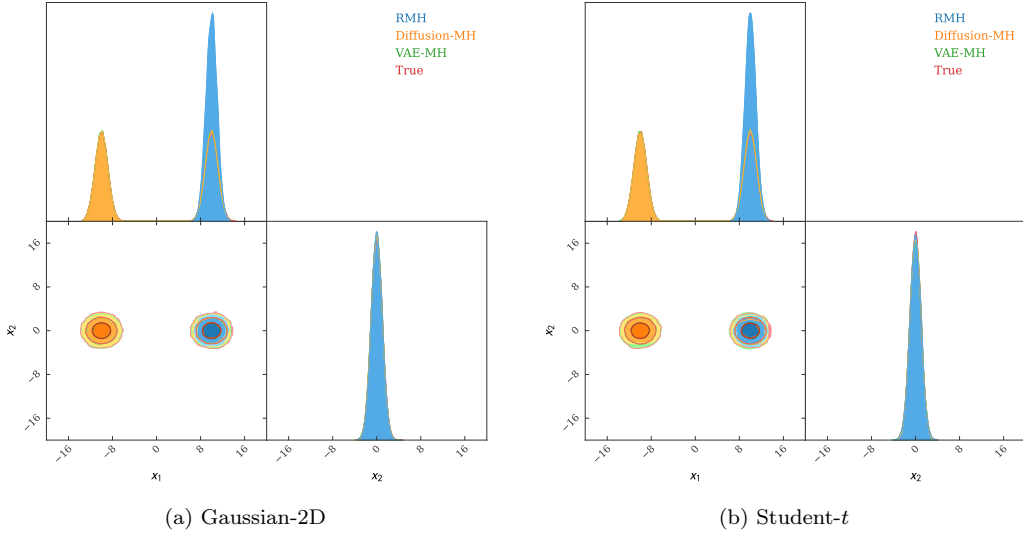


Figure 2: Corner plots of the RMH, Diffusion-MH, and VAE-MH algorithms for sampling the two-dimensional Gaussian and Student- t target distributions.

Table 5: Performance comparison of RMH, Diffusion-MH, and VAE-MH for sampling a 2-dimensional Gaussian mixture distribution.

Algorithm	Time (s)	ESS/s	Efficiency	Mode cov.	Mode TV $\downarrow$	Mode trans. $\uparrow$
RMH	17.20	NA	0.1165	1/2	0.5000	0.0000
Diffusion-MH	109.35	478.65	0.5785	2/2	0.0030	0.3658
VAE-MH	175.86	365.07	0.7098	2/2	0.0011	0.3961

Figure 2 and Tables 5–6 summarize the performance of the three samplers on the two-dimensional Gaussian and Student t mixture targets. The corner plots show that RMH has difficulty moving between the two separated modes. In the Gaussian mixture case, it remains in only one component, with zero

Table 6: Performance comparison of RMH, Diffusion-MH, and VAE-MH for sampling a 2-dimensional student-t mixture distribution. Reported values are averages over five independent runs.

Algorithm	Time (s)	ESS/s	Efficiency	Mode cov.	Mode TV ↓	Mode trans. ↑
RMH	18.30	NA	0.0552	1.8/2	0.3995	0.0000
Diffusion-MH	121.18	328.24	0.4414	2/2	0.0031	0.3572
VAE-MH	181.23	237.77	0.4741	2/2	0.0011	0.3267

mode-transition rate and a Mode TV distance of 0.5000. In the heavier-tailed Student t mixture, RMH occasionally reaches both modes across repeated runs, but its average mode-transition rate is still essentially zero, indicating poor cross-mode mixing.

Both Diffusion-MH and VAE-MH substantially improve global exploration. They cover both modes in both experiments and achieve very small Mode TV distances, showing that the empirical mode frequencies are close to the true mixture weights. Diffusion-MH attains the highest ESS/s in both examples, reflecting faster sampling per unit time. VAE-MH, on the other hand, achieves the highest sampling efficiency and the smallest Mode TV distance in both targets. This suggests that the learned VAE proposal is particularly effective at recovering the relative mass of separated modes, although it requires more computation than Diffusion-MH in these low-dimensional examples. Overall, the results confirm that global generative proposals can mitigate the mode-trapping behavior of random-walk MH, with VAE-MH providing the most accurate mode-frequency recovery among the three methods.

4.2.2. Sampling from a 10-dimensional Gaussian mixture

We next consider a two-component Gaussian mixture in ten dimensions, following the setting of Hunt-Smith et al. (2024). The target density is

$$f(\boldsymbol{\theta}) = w_A \frac{\exp\{-\|\boldsymbol{\theta} - \boldsymbol{\theta}_A\|^2/2\}}{(2\pi)^{d/2}} + w_B \frac{\exp\{-\|\boldsymbol{\theta} - \boldsymbol{\theta}_B\|^2/2\}}{(2\pi)^{d/2}},$$

where $d = 10$, $w_A = 2/3$, $w_B = 1/3$, $\boldsymbol{\theta}_A = (8, 3, \mathbf{0}_8^\top)^\top$, and $\boldsymbol{\theta}_B = (-2, 3, \mathbf{0}_8^\top)^\top$.

Table 7: Performance comparison of RMH, Diffusion-MH, and VAE-MH for sampling a 10-dimensional Gaussian mixture distribution. Reported values are averages over five independent runs.

Algorithm	Time (s)	ESS/s	Efficiency	Mode cov.	Mode TV $\downarrow$	Mode trans. $\uparrow$
RMH	18.32	NA	0.0249	1/2	0.6000	0.0000
Diffusion-MH	453.79	72.75	0.3638	2/2	0.0019	0.2739
VAE-MH	288.42	97.58	0.3127	2/2	0.0021	0.2221

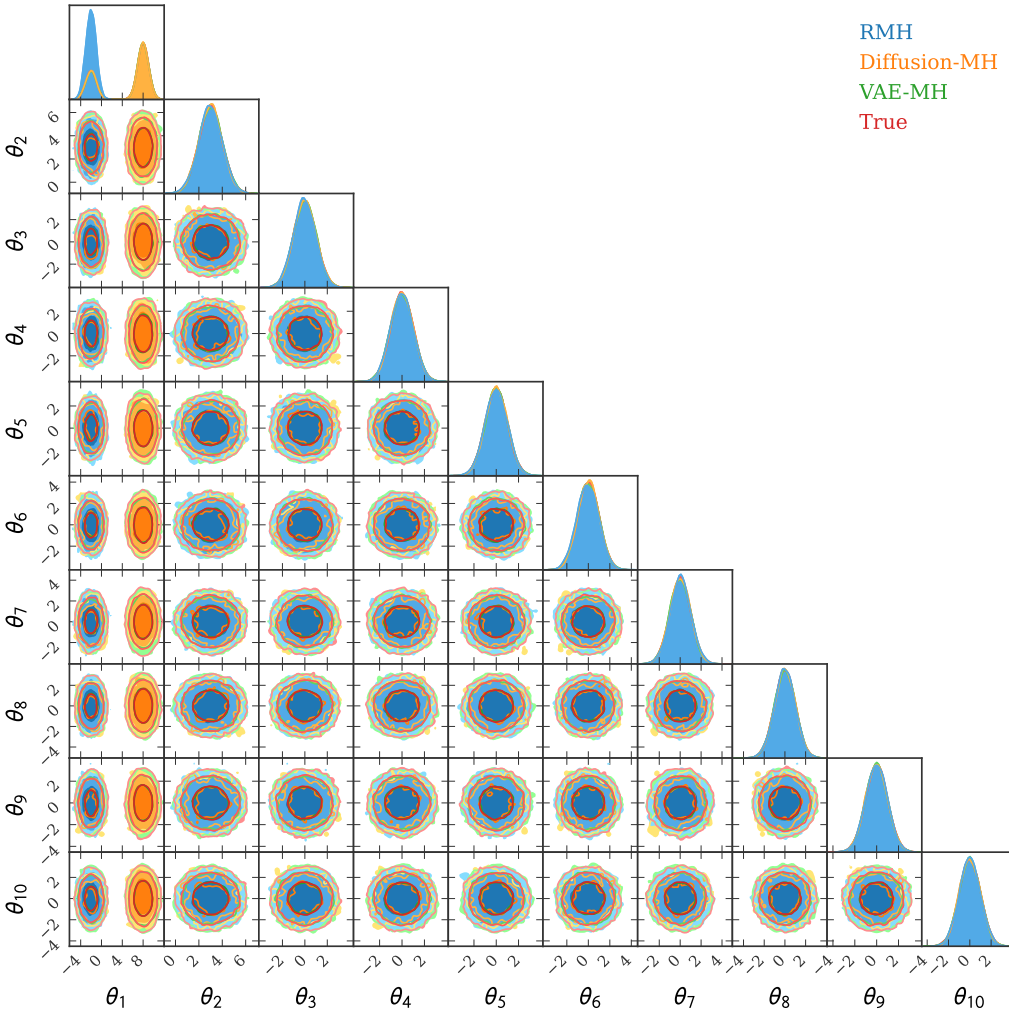


Figure 3: Corner plot of the RMH, Diffusion-MH, and VAE-MH algorithms for sampling the 10-dimensional Gaussian mixture distribution.

Figure 3 and Table 7 summarize the performance of the three samplers on the 10-dimensional Gaussian mixture. The corner plot shows that RMH samples only one component: the marginal distribution of θ_1 is unimodal, and the pairwise plots involving θ_1 do not show the second cluster. Both Diffusion-MH and VAE-MH recover the two components of the target distribution. Their contours in the corner plot are close to the true contours, and both methods achieve full mode coverage with small mode-TV distance. The main difference is computational. Diffusion-MH has a slightly higher efficiency and mode-transition rate, but it requires substantially longer run-time. VAE-MH attains a higher ESS per second while using less time, and its corner plot remains close to the target across the pairwise projections.

4.2.3. Sampling from 2-dimensional Gaussian mixture of 20 components

Following the experimental setup of Liang and Wong (2001), the target distribution is defined as:

$$f(\boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi}\sigma} \sum_{i=1}^{20} w_i \exp \left\{ -\frac{1}{2\sigma^2} (\boldsymbol{\theta} - \boldsymbol{\mu}_i)^\top (\boldsymbol{\theta} - \boldsymbol{\mu}_i) \right\},$$

where $\sigma = 0.1$, $w_1 = \dots = w_{20} = 0.05$, and the mean vectors $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_{20}$ are uniformly drawn from the rectangle $[0, 10] \times [0, 10]$, as listed in Table 8.

Table 8: Mean vectors of the 20 components of the mixture Gaussian distribution

i	μ_{i1}	μ_{i2}	i	μ_{i1}	μ_{i2}	i	μ_{i1}	μ_{i2}	i	μ_{i1}	μ_{i2}
1	2.18	5.76	6	3.25	3.47	11	5.41	2.65	16	4.93	1.50
2	8.67	9.59	7	1.70	0.50	12	2.70	7.88	17	1.83	0.09
3	4.24	8.48	8	4.59	5.60	13	4.98	3.70	18	2.26	0.31
4	8.41	1.68	9	6.91	5.81	14	1.14	2.39	19	5.54	6.86
5	3.93	8.82	10	6.87	5.40	15	8.33	9.50	20	1.69	8.11

20-Component Mixture Gaussian Distribution
($\sigma = 0.1$)

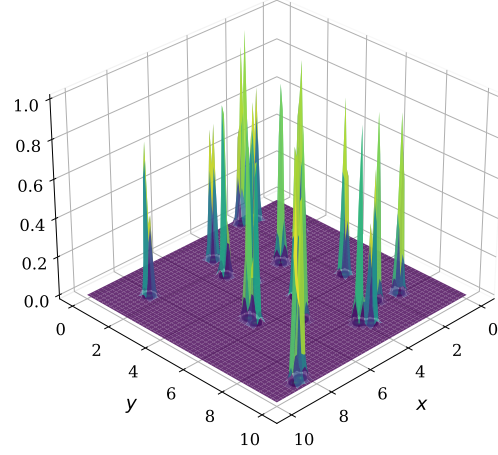


Figure 4: Density surface of the 20-component two-dimensional Gaussian mixture with equal component weights and $\sigma = 0.1$. The component means are located in $[0, 10]^2$, producing narrow isolated peaks separated by regions with negligible probability mass.

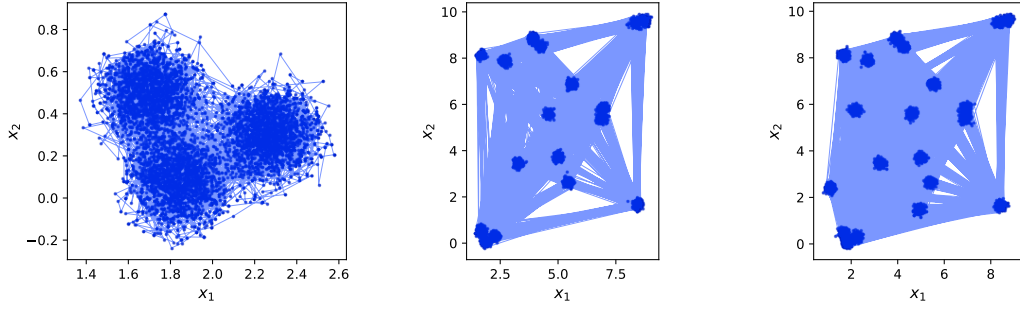


Figure 5: Trajectory plot of the first 10000 samples of random-walk MH, Diffusion-MH and VAE-MH algorithms.

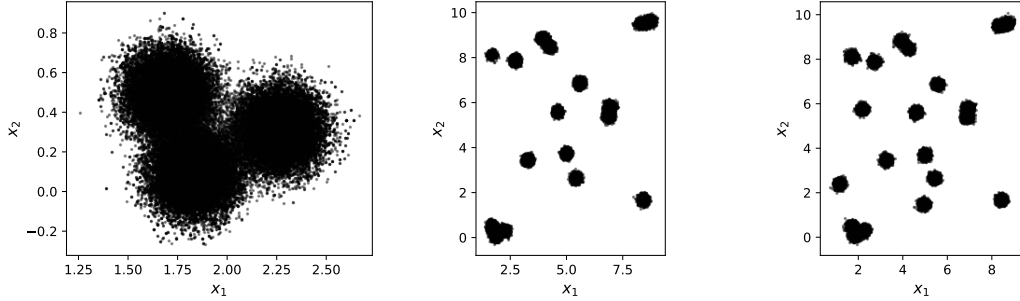


Figure 6: The plot of whole random-walk MH samples, Diffusion-MH samples and VAE-MH samples.

Table 9: Performance comparison of RMH, Diffusion-MH, and VAE-MH for sampling a 2-dimensional Gaussian mixture distribution with 20 components. Reported values are averages over five independent runs.

Algorithm	Time (s)	ESS/s	Efficiency	Mode cov.	Mode TV ↓	Mode trans. ↑
RMH	16.91	125.84	0.0236	3/20	0.8500	0.0548
Diffusion-MH	92.86	192.36	0.1985	19/20	0.1905	0.3490
VAE-MH	196.94	293.76	0.6428	20/20	0.0076	0.7587

Figures 5 and 6 compare the mixing behavior of the three samplers. The random-walk MH sampler stays in a local region for a long period and does not move effectively between separated components. Both Diffusion-MH and VAE-MH improve cross-mode exploration, Diffusion-MH visits 18 out of the 20 components, whereas VAE-MH recovers all 20 components. The trajectory plot also shows better and more stable transitions for VAE-MH, indicating better mixing across the separated modes. The numerical results in Table 9 are consistent with these visual findings. RMH has the lowest runtime, but its low efficiency and poor mode coverage show that it is not sampling the full target distribution adequately. Diffusion-MH improves global movement, but it misses two components with a larger computational cost. VAE-MH achieves best overall performance with a full mode coverage, a smaller mode-frequency error and better mixing behavior.

5. Discussions

There are two main innovative points in this paper.

1. We innovatively propose to utilize GVI to derive a covariance matrix for the linear transformation in HMC. What is more, we use GVI with various covariance structures. Specifically, we consider GVI with Cholesky decomposed covariance Tran et al. (2021), GVI with factor decomposed covariance Ong et al. (2018) and GVI with sparse precision matrices Tan and Nott (2018). These methods address the challenge of estimating covariance matrices for the linear transformation in HMC when the distribution is complex and high-dimensional.
2. We propose to combine a VI-based generative model with the random-walk MH sampler by training a VAE-type model on mode-informed and adaptively updated samples, generating a global proposal pool, and applying an MH correction through an explicit KDE proposal density. The implementation includes standard VAEs, flow-augmented VAEs, component-conditioned GMM-VAEs, and Student- t decoder variants for heavy-tailed targets. This innovative "VAE-MH" sampler is straightforward to implement and more efficient in traversing the whole parameter space than state-of-the-art MCMC algorithms.

In future work, we will investigate the possibility of combining density-matching generative models that do not require training samples with advanced MCMC algorithms, thereby reducing the dependence on samples from the target distribution. Furthermore, we will explore the combination of various generative models with more advanced MCMC algorithms, including the reversible jump MCMC Green (1995), which is a generalization of MH and allows jumping among parameter spaces with varying dimensions.

6. Supplementary materials

References

- Beskos, A., Pillai, N., Roberts, G., Sanz-Serna, J.M., Stuart, A., 2013. Optimal tuning of the hybrid Monte Carlo algorithm. *Bernoulli. Official Journal of the Bernoulli Society for Mathematical Statistics and Probability* 19, 1501–1534. doi:10.3150/12-BEJ414.
- Blei, D.M., Kucukelbir, A., McAuliffe, J.D., 2017. Variational inference: A review for statisticians. *Journal of the American Statistical Association* 112, 859–877. doi:10.1080/01621459.2017.1285773.

- Chauhan, D., Shivani, Jung, D., Yadav, A., 2025. Advancements in multimodal differential evolution: A comprehensive review and future perspectives. *Artificial Intelligence Review* 58, 335. doi:10.1007/s10462-025-11314-7.
- Gabri , M., Rotskoff, G.M., Vanden-Eijnden, E., 2021. Efficient Bayesian sampling using normalizing flows to assist Markov chain Monte Carlo methods, in: ICML Workshop on Invertible Neural Networks, Normalizing Flows, and Explicit Likelihood Models. URL: <https://openreview.net/forum?id=mvtooHbj0wx>.
- Gelman, A., Gilks, W.R., Roberts, G.O., 1997. Weak convergence and optimal scaling of random walk metropolis algorithms. *The Annals of Applied Probability* 7, 110–120. doi:10.1214/aoap/1034625254.
- Geyer, C.J., 1991. Markov chain Monte Carlo maximum likelihood URL: <https://hdl.handle.net/11299/58440>, arXiv:11299/58440.
- Girolami, M., Calderhead, B., 2011. Riemann manifold langevin and hamiltonian monte carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 73, 123–214. doi:10.1111/j.1467-9868.2010.00765.x.
- Green, P.J., 1995. Reversible jump Markov chain Monte Carlo computation and Bayesian model determination. *Biometrika* 82, 711–732. doi:10.1093/biomet/82.4.711.
- Haario, H., Saksman, E., Tamminen, J., 1999. Adaptive proposal distribution for random walk Metropolis algorithm. *Computational Statistics* 14, 375–395. doi:10.1007/s001800050022.
- Haario, H., Saksman, E., Tamminen, J., 2001. An adaptive Metropolis algorithm. *Bernoulli. Official Journal of the Bernoulli Society for Mathematical Statistics and Probability* 7, 223–242. doi:10.2307/3318737.
- Habib, R., Barber, D., 2019. Auxiliary variational MCMC, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=r1NJqsRctX>.

- Hastings, W.K., 1970. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika* 57, 97–109. doi:10.1093/biomet/57.1.97.
- Hunt-Smith, N.T., Melnitchouk, W., Ringer, F., Sato, N., Thomas, A.W., White, M.J., 2024. Accelerating markov chain monte carlo sampling with diffusion models. *Computer Physics Communications* 296, 109059. doi:10.1016/j.cpc.2023.109059.
- Kingma, D.P., Welling, M., 2014. Auto-encoding variational bayes, in: *International Conference on Learning Representations*. URL: <https://arxiv.org/abs/1312.6114>.
- Lan, S., Streets, J., Shahbaba, B., 2014. Wormhole hamiltonian monte carlo, in: *Proceedings of the AAAI Conference on Artificial Intelligence*. doi:10.1609/aaai.v28i1.9006.
- Lavda, F., Gregorová, M., Kalousis, A., 2020. Improving VAE generations of multimodal data through data-dependent conditional priors, in: *Proceedings of the 24th European Conference on Artificial Intelligence*, IOS Press. pp. 1254–1261. doi:10.3233/FAIA200226.
- Liang, F., Wong, W.H., 2001. Real-parameter evolutionary monte carlo with applications to bayesian mixture models. *Journal of the American Statistical Association* 96, 653–666. doi:10.1198/016214501753168325.
- Marinari, E., Parisi, G., 1992. Simulated tempering: A new Monte Carlo scheme. *Europhysics letters* 19, 451. doi:10.1209/0295-5075/19/6/002.
- Metropolis, N., Rosenbluth, A.W., Rosenbluth, M.N., Teller, A.H., Teller, E., 1953. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics* 21, 1087–1092. doi:10.1063/1.1699114.
- Nalisnick, E., Hertel, L., Smyth, P., 2016. Approximate inference for deep latent Gaussian mixtures, in: *NeurIPS Workshop on Bayesian Deep Learning*. URL: https://bayesiandeeplearning.org/2016/papers/BDL_20.pdf.
- Neal, R.M., 2011. MCMC using Hamiltonian dynamics, in: *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, pp. 113–162. URL: <https://www.taylorfrancis.com/chapters/edit/10.1201/b10905-6>.

- Ong, V.M.H., Nott, D.J., Smith, M.S., 2018. Gaussian variational approximation with a factor covariance structure. *Journal of Computational and Graphical Statistics* 27, 465–478. doi:10.1080/10618600.2017.1390472.
- Park, J., 2021. Sampling from multimodal distributions using tempered Hamiltonian transitions. URL: <https://arxiv.org/abs/2111.06871v1>.
- Rezende, D.J., Mohamed, S., 2015a. Variational inference with normalizing flows, in: *Proceedings of the 32nd International Conference on Machine Learning*, pp. 1530–1538. URL: <https://proceedings.mlr.press/v37/rezende15.html>.
- Rezende, D.J., Mohamed, S., 2015b. Variational inference with normalizing flows, in: *Proceedings of the 32nd International Conference on Machine Learning*, PMLR. pp. 1530–1538.
- Robert, C.P., Casella, G., 2004. *Monte Carlo Statistical Methods*. Springer Texts in Statistics. second ed., Springer. URL: <https://doi.org/10.1007/978-1-4757-4145-2>.
- Roberts, G.O., Rosenthal, J.S., 1998. Optimal scaling of discrete approximations to Langevin diffusions. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 60, 255–268. doi:10.1111/1467-9868.00123.
- Samsonov, S., Lagutin, E., Gabri’e, M., Durmus, A., Naumov, A., Moulines, E., 2022. Local-global mcmc kernels: The best of both worlds, in: *Advances in Neural Information Processing Systems*, pp. 5178–5193. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/21c86d5b10cdc28664ccdadf0a29065a-Abstract-Conference.html.
- Storn, R., Price, K., 1997. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization* 11, 341–359. doi:10.1023/A:1008202821328.
- Tak, H., Meng, X.L., van Dyk, D.A., 2018. A repelling–attracting Metropolis algorithm for multimodality. *Journal of Computational and Graphical Statistics* 27, 479–490. doi:10.1080/10618600.2017.1415911.
- Tan, L.S.L., Nott, D.J., 2018. Gaussian variational approximation with sparse precision matrices. *Statistics and Computing* 28, 259–275. doi:10.1007/s11222-017-9729-7.

- Tan, P.L., Peharz, R., 2019. Hierarchical decompositional mixtures of variational autoencoders, in: Proceedings of the 36th International Conference on Machine Learning, PMLR. pp. 6115–6124.
- Thomsen, R., 2004. Multimodal optimization using crowding-based differential evolution, in: Proceedings of the 2004 congress on evolutionary computation (IEEE cat. no.04th8753), pp. 1382–1389 Vol.2. doi:10.1109/CEC.2004.1331058.
- Tran, M.N., Nguyen, T.N., Dao, V.H., 2021. A practical tutorial on variational Bayes. URL: <https://arxiv.org/abs/2103.01327>.
- Ye, F., Bors, A.G., 2020. Mixtures of variational autoencoders, in: 2020 Tenth International Conference on Image Processing Theory, Tools and Applications, IEEE, Paris, France. pp. 1–6. doi:10.1109/IPTA50016.2020.9286619.
- Zhang, C., Matsen IV, F.A., 2019. Variational Bayesian phylogenetic inference, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=SJVmjR9FX>.

Algorithm 1 GVI-preconditioned Hamiltonian Monte Carlo

```

1: Obtain  $q_{\text{GVI}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}_{\text{GVI}}, \boldsymbol{\Sigma}_{\text{GVI}})$  by GVI
2: Set  $\mathbf{S} \leftarrow \boldsymbol{\Sigma}_{\text{GVI}}$  and initialize  $\boldsymbol{\theta} \leftarrow \boldsymbol{\mu}_{\text{GVI}}$ 
3: Compute  $\ell \leftarrow \ell(\boldsymbol{\theta}) = \log \pi(\boldsymbol{\theta})$ 
4: Compute an upper-triangular Cholesky factor  $\mathbf{R} = \text{chol}(\mathbf{S}^{-1})$ , so that
    $\mathbf{R}^\top \mathbf{R} = \mathbf{S}^{-1}$ 
5: for  $t = 1$  to  $B + N$  do
6:   Draw  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$  and set  $\boldsymbol{\psi} \leftarrow \mathbf{R}^\top \mathbf{z}$   $\triangleright \boldsymbol{\psi} \sim \mathcal{N}(\mathbf{0}, \mathbf{S}^{-1})$ 
7:   Compute  $K \leftarrow \frac{1}{2} \boldsymbol{\psi}^\top \mathbf{S} \boldsymbol{\psi}$ 
8:   Set  $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$  and  $\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi}$ 
9:   
$$\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi}^* + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}^*)$$

    $\triangleright$  Half step for momentum
10:  for  $l = 1$  to  $L$  do
11:    
$$\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}^* + \epsilon \mathbf{S} \boldsymbol{\psi}^*$$

     $\triangleright$  Full step for position
12:    if  $l \neq L$  then
13:      
$$\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi}^* + \epsilon \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}^*)$$

       $\triangleright$  Full step for momentum
14:      
$$\boldsymbol{\psi}^* \leftarrow \boldsymbol{\psi}^* + \frac{\epsilon}{2} \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}^*)$$

       $\triangleright$  Half step for momentum
15:      
$$\boldsymbol{\psi}^* \leftarrow -\boldsymbol{\psi}^*$$

       $\triangleright$  Momentum flip for reversibility
16:      Compute  $\ell^* \leftarrow \ell(\boldsymbol{\theta}^*) = \log \pi(\boldsymbol{\theta}^*)$  and  $K^* \leftarrow \frac{1}{2} (\boldsymbol{\psi}^*)^\top \mathbf{S} \boldsymbol{\psi}^*$ .
17:      Compute  $\log a \leftarrow \ell^* - \ell + K - K^*$ .
18:      Draw  $u \sim \text{Uniform}(0, 1)$ 
19:      if  $\log u < \min\{0, \log a\}$  then
20:         $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^*$ 
21:         $\ell \leftarrow \ell^*$ 
22:      if  $t > B$  then
23:        Store  $\boldsymbol{\theta}^{(t-B)} \leftarrow \boldsymbol{\theta}$ 

```

Algorithm 2 Crowding-based Differential Evolution Algorithm

Input: $f(\mathbf{x})$: Multimodal objective function

F : Mutation factor

CR : Crossover rate

N : Population size

G_{max} : Maximum generations

Output: $P^{G_{max}}$: Final population

```
1:  $\triangleright$  Initialization  $\triangleleft$ 
2: Set generation counter  $G \leftarrow 0$ 
3: Randomly initialize population  $P^0 = \{\mathbf{x}_1^0, \mathbf{x}_2^0, \dots, \mathbf{x}_N^0\}$ 
4:  $\triangleright$  Main Loop  $\triangleleft$ 
5: while  $G \leq G_{max}$  do
6:   for  $i = 1$  to  $N$  do
7:     procedure MUTATION
8:        $\triangleright$  DE/rand/1  $\triangleleft$ 
9:       Randomly select  $r_1, r_2, r_3 \in \{1, 2, \dots, N\}$  with  $r_1 \neq r_2 \neq r_3 \neq i$ 
10:       $\mathbf{v}_i^G \leftarrow \mathbf{x}_{r_1}^G + F \cdot (\mathbf{x}_{r_2}^G - \mathbf{x}_{r_3}^G)$ 
11:     procedure CROSSOVER
12:        $\triangleright$  Ensure at least one component from mutant  $\triangleleft$ 
13:        $j_{rand} \leftarrow U\{1, D\}$   $\triangleright$  Randomly select
14:       for  $j = 1$  to  $D$  do
15:         if  $\text{rand}() \leq CR$  or  $j = j_{rand}$  then
16:            $u_{i,j}^G \leftarrow v_{i,j}^G$   $\triangleright$  Inherit from mutation vector
17:         else
18:            $u_{i,j}^G \leftarrow x_{i,j}^G$   $\triangleright$  Inherit from target vector
19:     procedure SELECTION
20:        $\mathbf{x}_{closest}^G \leftarrow \arg \min_{\mathbf{x} \in P^G} \|\mathbf{x} - \mathbf{u}_i^G\|$   $\triangleright$  Find closest individual
21:       if  $f(\mathbf{u}_i^G) \geq f(\mathbf{x}_{closest}^G)$  then
22:          $\mathbf{x}_{closest}^G \leftarrow \mathbf{u}_i^G$ 
23:    $G \leftarrow G + 1$   $\triangleright$  Increment generation counter
```

Algorithm 3 Generative-proposal MH sampler for multi-modal targets

- 1: Identify approximate modes $\hat{\mathbf{m}}_1, \dots, \hat{\mathbf{m}}_J$ using CrowdingDE
 - 2: Construct the initial training set $\mathcal{D}_0$ from local samples around the estimated modes
 - 3: Initialize the chain at a point in $\mathcal{D}_0$
 - 4: **for** $r = 0, 1, \dots, R - 1$ **do**
 - 5: Train a VAE-type generative model G_r on $\mathcal{D}_r$
 - 6: Generate proposal pool $\mathcal{Y}_r = \{\mathbf{y}_1, \dots, \mathbf{y}_M\}$ from G_r
 - 7: Fit the KDE proposal density q_r in equation (3)
 - 8: **for** $s = 1, \dots, \tau$ **do**
 - 9: Draw $u \sim U(0, 1)$
 - 10: **if** $u < \rho$ **then**
 - 11: Propose $\boldsymbol{\theta}'$ from the KDE proposal q_r
 - 12: Accept $\boldsymbol{\theta}'$ with probability in equation (4)
 - 13: **else**
 - 14: Propose $\boldsymbol{\theta}' = \boldsymbol{\theta}^{(t)} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{rw}}^2 \mathbf{I})$
 - 15: Accept $\boldsymbol{\theta}'$ with random-walk MH probability
 - 16: Append the current state to the chain
 - 17: Update $\mathcal{D}_{r+1}$ using the most recent training window of chain states
-